

Практическая работа №4

Рефакторинг и тестирование .NET микросервисов

Цели и задачи

Краткая выжимка из задания и фактически выполненных работ.

Архитектура

- провести анализ исходного кода

- выявить нарушения clean architecture, SOLID и best practices

- разделить ответственность Library и Reports

Код и платформа

- перевести решения с .NET 8 на .NET 10

- обновить зависимости до совместимых версий

- устранить проблемы restore/build

Качество

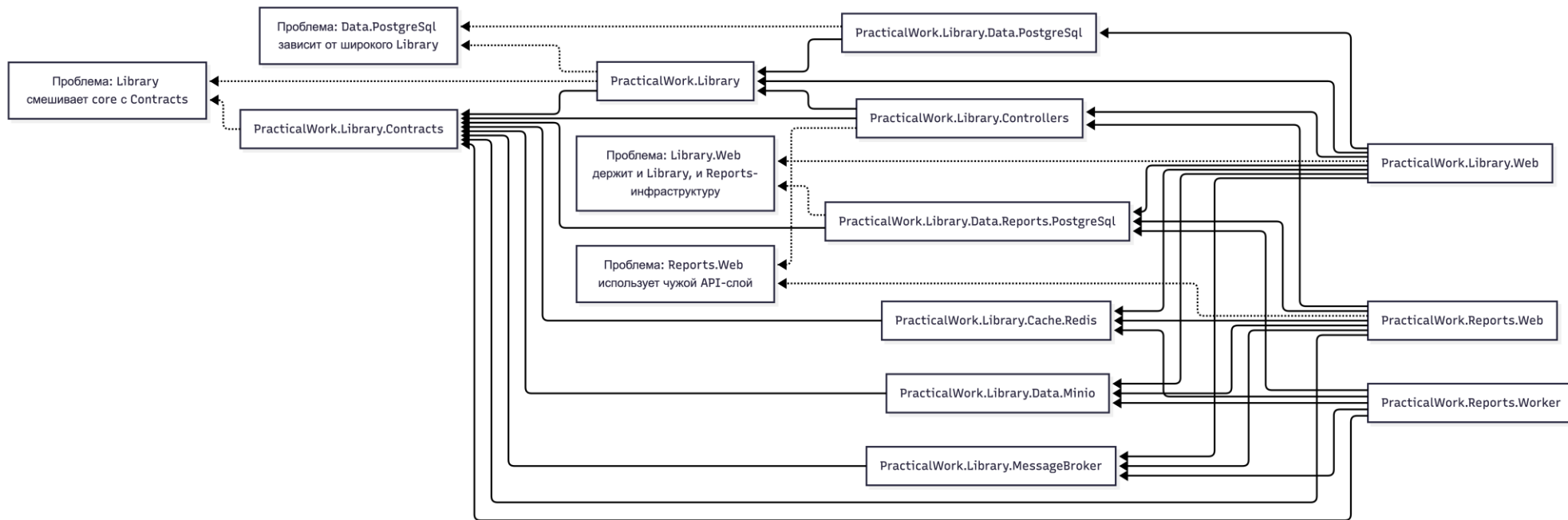
- добавить unit-тесты для бизнес-логики и интеграций

- измерить покрытие по Application/Domain и Infrastructure

- подготовить документацию и презентацию

Технология	Назначение	Версия / состояние
.NET	бэкенд платформа	10.0
PostgreSQL, Redis, MinIO	данные, кэш, файлы	используются через adapters
RabbitMQ, Hangfire, MailKit	сообщения, фоновые задачи, email	подключены в проектах
xUnit, Moq, coverlet/TestPlatform	unit-тесты и coverage	70 тестов проходят

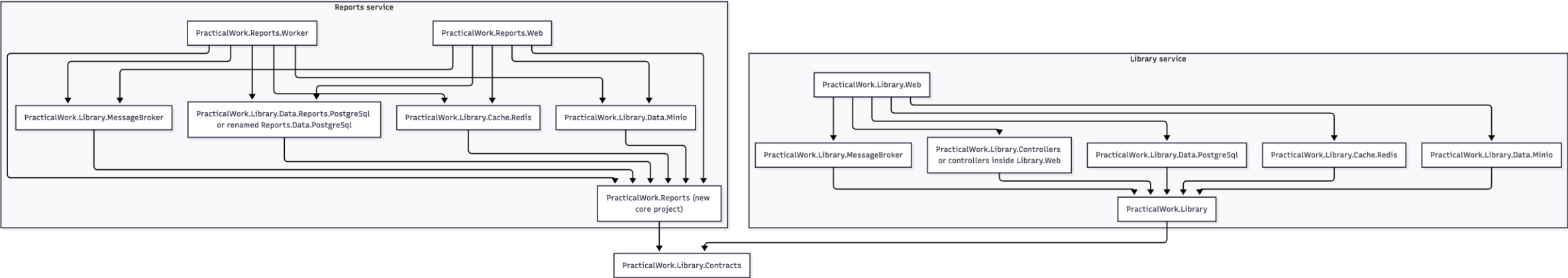
Исходная архитектура и проблемные места



Зафиксированные проблемы

- Library смешивает core с Contracts; Contracts содержит не только внешние контракты.
- Library.Web держит и библиотечную, и reports-инфраструктуру.
- Reports.Web использует чужой API-слой через Library.Controllers.
- Data.PostgreSql зависит от широкого PracticalWork.Library.

Целевая архитектура чистых границ



Library service

- книги, читатели, выдача и возврат
- собственный Web/API слой
- свои adapters: PostgreSQL, cache, MinIO, broker

Reports service

- журнал событий и CSV-отчёты
- Reports.Web и Reports.Worker
- выделенный core PracticalWork.Reports

Contracts

- общие integration events
- публичные DTO, если они действительно общие
- взаимодействие без зависимости от runtime Library

Архитектурные изменения

Было	Стало	Архитектурный эффект
Reports.* -> PracticaWork.Library	Reports.* -> PracticaWork.Library.Contracts	Reports не тянет runtime-сборку Library
Contracts зависел от Library	Contracts стал нейтральной сборкой	контракты используются как внешний слой взаимодействия
Report-инфраструктура брала типы из Library	Data.Reports, MinIO, Cache, Broker используют Contracts	инфраструктура Reports отвязана от домена Library
Reports.Web использовал Library.Controllers	Reports.Web содержит собственный ReportController	presentation-граница микросервиса стала самостоятельной
Что подтвердилось в коде - В PracticalWork.Reports.Web.csproj и PracticalWork.Reports.Worker.csproj нет прямого ProjectReference на PracticalWork.Library.csproj. - PracticalWork.Library.Data.Reports.PostgreSql зависит от PracticalWork.Library.Contracts. - Reports.Web содержит собственные Controllers/ReportController.cs, MvcSetupExtensions.cs и validators.		

Контроллер: было / стало

ReportController вынесен из библиотечного API в Reports.Web.

Было: PracticalWork.Library.Controllers

```
using PracticalWork.Library.Abstractions.Services;
using PracticalWork.Library.Controllers.Mappers.v1;

public async Task<IActionResult> GetActivityLogs(...)
{
    var result = await _reportService.ReadSystemActivityLogs(
        request.ToGetActivityLogsRequestModel());
    return Ok(result.ToActivityLogsPaginationResponse());
}
```

Стало: PracticalWork.Reports.Web.Controllers

```
using PracticalWork.Library.Contracts.Abstractions.Services;
using PracticalWork.Library.Contracts.v1.Reports.Request;

public async Task<IActionResult> GetActivityLogs(...)
{
    return Ok(await _reportService.ReadSystemActivityLogs(request));
}

public async Task<IActionResult> CreateReportCsv(...)
{
    return Ok(await _reportService.CreateReport(request));
}
```

Архитектурный и кодовый результат

- Reports API больше не публикуется через Library.Controllers.
- Reports.Web сам настраивает API versioning, discovery контроллеров и FluentValidation.
- Контроллер стал тоньше: он принимает request и передаёт его в contract service без library mappers.

Репозиторий: было / стало

ReportRepository и DI-регистрация переведены на Contracts.

Было: зависимость от PracticalWork.Library

```
using PracticalWork.Library.Abstractions.Storage;
using PracticalWork.Library.Models.ReportModels;
using PracticalWork.Library.Enums;
using PracticalWork.Library.Exceptions;

serviceCollection.AddScoped<IReportRepository, ReportRepository>();
```

Стало: зависимость от PracticalWork.Library.Contracts

```
using PracticalWork.Library.Contracts.Abstractions.Storage;
using PracticalWork.Library.Contracts.Models.ReportModels;
using PracticalWork.Library.Contracts.Exceptions;

serviceCollection.AddScoped<IReportRepository, ReportRepository>();
```

Элемент	Архитектурный момент	Кодовый момент
ReportRepository	data layer Reports не зависит от домена Library	типы заменены на Contracts.*
AddReportsPostgreSqlStorage	DI регистрирует report storage отдельно	IActivityLogRepository, IReportRepository из Contracts
ReportsDbContext	остаётся инфраструктурной деталью Reports	не протекает в контроллеры

Фоновая задача: было / стало

ReportConsumerWorker и обработчики событий переведены на contract model.

Было: worker тянет Library runtime

```
Reports.Worker -> PracticalWork.Library
```

```
using PracticalWork.Library.Events;  
using PracticalWork.Library.Models.ReportModels;  
using PracticalWork.Library.Abstractions.Storage;
```

Фоновая логика привязана к чужому bounded context.

Стало: worker работает через Contracts

```
Reports.Worker -> PracticalWork.Library.Contracts
```

```
using PracticalWork.Library.Contracts.Events;  
using PracticalWork.Library.Contracts.Abstractions.Storage;  
using PracticalWork.Reports.Worker.Models;
```

ReportGenerateResult локален для Reports.Worker.

Архитектурный момент

- событие ReportCreateEvent живёт в нейтральном contract layer
- Reports.Worker не использует report models из PracticalWork.Library
- фоновые сценарии отчётов отделены от Library domain

Кодовый момент

- обработчики используют Contracts.Abstractions.MessageBroker
- worker создаёт scope и вызывает report handler
- результат генерации отчёта хранится в локальной модели worker

Структура unit-тестов

21

test-файл

70

тестов проходит

40

Library.Tests

30

Reports.Tests

0

failed

Проект	Группы тестов	Что покрывается
PracticalWork.Library.Tests	Services, Models, Handlers	сервисы книг/читателей/выдачи, архивирование, уведомления, weekly report
PracticalWork.Reports.Tests	Controllers, Services, Validations, Worker, Contracts	Reports API, валидация, генерация отчётов, обработчики событий
TestCommon	StaticOptionsMonitor, TestTimeProvider	общие test helpers для изоляции времени и options

- проверяются успешные сценарии, невалидные данные, not found, конфликтные ситуации и вызовы зависимостей
- фоновые задачи проверяются без реального ожидания; email/report сценарии проверяются без реальной отправки
- для повторяемости используются xUnit, Moq, coverlet.collector и локальные test helpers

Результаты покрытия тестами

Покрытие измерено отдельными фильтрами: Application/Domain.



Фильтр	Команда	HTML отчёт
Application/Domain	coverage.app-domain.runsettings	artifacts/coverage-report/app-domain/index.html

Процесс миграции на .NET 10

12

проектов переведены на net10.0

10.0.201

SDK проверки

0 / 0

warnings / errors в build

10.0.0

EF Core, Npgsql, Microsoft.*

Этап	Что сделано	Результат
TargetFramework	все .csproj переведены с net8.0 на net10.0	артефакты собираются в bin/Debug/net10.0
Central packages	Microsoft.*, EF Core, Npgsql выровнены на 10.x	совместимые версии в Directory.Packages.props
NU1903	удалены лишние EF Tools/Design из runtime data library	restore и build проходят без ошибок
Dockerfiles	web/worker/migrator Dockerfile обновлены под .NET 10	контейнерная сборка соответствует платформе

Созданная документация

Материалы покрывают эксплуатацию, архитектуру, рефакторинг, тестирование и миграцию.

Файл / артефакт	Назначение
README.md, init.md	структура решения, запуск, инфраструктура, API routes, Docker Compose
XML docs, Swagger	документация публичного API Library.Web и Reports.Web

Выводы

Архитектура и код

- границы Library и Reports разделены лучше
- Reports.* переведены на contract-only взаимодействие
- контроллер, репозиторий и worker показаны в before/after формате

Качество и платформа

- проект мигрирован на .NET 10
- dotnet build проходит без предупреждений и ошибок
- 70 unit-тестов проходят успешно

Покрытие и отчёты

- покрытие измерено отдельно для Application/Domain
- HTML-отчёты сформированы через reportgenerator

Документация

- подготовлены README
- Swagger и XML documentation настроены для API